

Supplemental Material: Physics-Exact Credit Assignment for Multi-Agent Flutter Suppression

This document collects material referenced from the main text but moved here to meet the journal’s page-length guidance. Section, table, and figure numbers below are internal to this document (prefixed S) and are cross-referenced from the main text by name, not by number.

S1. Network architecture, training procedure, and convergence

Network architecture

Figure 1 shows the full policy pipeline. Every variant feeds agent k ’s local observation $o_k^t \in \mathbb{R}^{12}$, concatenated with a one-hot agent identity and (when enabled) an incoming consensus message e_k^t , through a two-layer tanh multilayer perceptron of width 128 to a latent ℓ_k^t . Parameters are shared across agents; only the one-hot identity varies, which keeps the parameter count independent of the number of surfaces and is what makes the eight-surface scaling probe (Section S5) possible without retraining a wider network.

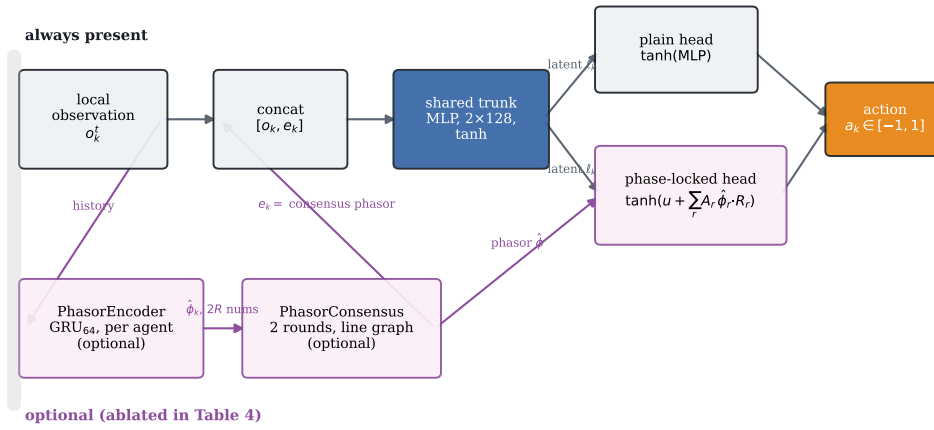


Figure 1: The policy network. Grey boxes and the plain head are present in every variant; the pink boxes and the phase-locked head are the mechanisms ablated in the main text’s architecture table, so **recurrent_only** keeps the encoder but not consensus or the head, **mocca_nohead** keeps the encoder and consensus but ends in the plain head, and **mocca** is the full diagram.

Phasor encoder (recurrent_only and beyond) A single instantaneous accelerometer reading cannot distinguish a mode moving up from the same displacement moving down, so recovering

phase needs memory. A per-agent GRU cell (Hausknecht and Stone, 2015) carries a hidden state $h_k^t \in \mathbb{R}^{64}$ across the episode:

$$h_k^t = \text{GRU}(o_k^t, h_k^{t-1}), \quad \hat{\phi}_k^t = \tanh(W_{\text{out}} h_k^t) \in \mathbb{R}^4, \quad (1)$$

read as two pairs $(\hat{\phi}_{k,r}^{\cos}, \hat{\phi}_{k,r}^{\sin})$, one per retained mode. The output is passed through $\tanh$ before use anywhere downstream: an unbounded phasor multiplied by a gain inside the action head saturates that head’s own $\tanh$ at initialisation and kills the gradient before training starts, which is what the first implementation of this policy did. The phasor is carried as a (cos, sin) pair rather than an angle throughout, so wrap-around is not a discontinuity the network has to fight.

Phasor consensus (mocca and comm_raw) The communication mechanism is $T=2$ rounds of averaging over the spanwise line graph – agent k exchanges only with agents $k-1$ and $k+1$, what a distributed avionics bus physically offers. The operator

$$P = (1 - w) I + w A, \quad w = 0.5, \quad (2)$$

with A the row-normalised adjacency of the line graph, is fixed rather than learned, so consensus adds no parameters; the bandwidth cost is the message itself, 4 numbers per agent per round regardless of surface count. The consensus message is $e^t = P^T \hat{\phi}^t$, read back into the trunk of each agent as e_k^t . **comm_raw** replaces $\hat{\phi}_k^t$ in this pipeline with the agent’s raw local observation instead of the encoder’s phasor estimate, testing whether the phasor abstraction itself does anything relative to simply broadcasting what a neighbour sees.

Phase-locked action head (the full mocca architecture) For retained mode r with consensus phasor $(\hat{\phi}_r^{\cos}, \hat{\phi}_r^{\sin})$, the head emits a gain $A_r \in [-1, 1]$ and a phase offset via a unit vector $(\cos \psi_r, \sin \psi_r)$, and acts by rotating the phasor:

$$\rho_r = \hat{\phi}_r^{\cos} \cos \psi_r - \hat{\phi}_r^{\sin} \sin \psi_r, \quad a_k = \tanh\left(u + \sum_r A_r \rho_r\right), \quad (3)$$

where u is an additional broadband channel from the same latent. The resonant term is *added* to u rather than replacing it, a strict generalisation of the plain head: setting every A_r to zero recovers an unconstrained policy exactly. An earlier version of this head replaced u instead of adding to it, turning a resonant prior into a restriction, and scored worse than removing the head entirely – **mocca_nohead** is the trunk and consensus without this head, i.e. the plain $\tanh(\text{MLP})$ output at the top of Figure 1.

Critic The critic is a separate two-layer MLP taking the same per-agent input as the actor by default; a fully centralised critic (fed the true modal state instead) was tried and made no measurable difference, consistent with this being a dense-reward, low-partial-observability credit problem rather than one where value estimation is the bottleneck. Every head has its final layer scaled by 0.01 at initialisation so a fresh policy is close to a no-op, which is what makes the residual variants well-posed from the first update rather than starting by fighting the base controller.

Training procedure

We use shared-parameter multi-agent PPO (Schulman et al., 2017; Yu et al., 2022) with generalised advantage estimation (Schulman et al., 2016) and per-agent advantages, since each agent receives a

different reward. Updates run on sequences: minibatches are segments of consecutive steps unrolled with gradient from a detached stored hidden state. Sampling shuffled individual timesteps, as we did initially, trains any recurrence as a one-step map and gives a recurrent encoder no temporal gradient at all (Hausknecht and Stone, 2015). Algorithm 1 summarises the procedure.

Algorithm 1 Training with the blended (exact-plus-shaping) credit

```

1: Assemble  $M, C, K, L, B$  at the sampled airspeed; small-gain init of the action head
2: for each update do
3:   Ease the task distribution by the curriculum fraction
4:   for  $t = 1, \dots, T$  do
5:     Each agent  $k$  acts on its own  $o_k^t$ ; step the coupled plant and actuators
6:      $P_k^t \leftarrow (\dot{q}^\top b_k) \delta_k^t$   $\triangleright$  closed form
7:      $r_k^t \leftarrow -P_k^t / (E^t + \varepsilon) + w [\Phi^{t+1} - \Phi^t] / \Delta t$   $\triangleright \Phi = -\log \frac{E+\varepsilon}{E_{\text{ref}}}$ 
8:   end for
9:   Compute per-agent GAE advantages
10:  for each epoch, each minibatch of sequences do
11:    Unroll  $L_{\text{bptt}}$  steps from a detached hidden state; zero it where an episode ended
12:    PPO clipped objective + value loss – entropy bonus
13:  end for
14: end for
```

Hyperparameters

Table 1 lists the PPO and network hyperparameters shared across every learned variant, except where a hyperparameter is itself the subject of an ablation (the initial exploration standard deviation, and the presence or absence of the architectural mechanisms above).

Compute and reproducibility

Every learned variant trains for 1.5×10^6 environment steps across 128 vectorised environments on a single NVIDIA RTX 5000 Ada (laptop) GPU, using PyTorch 2.6. Across the 86 training runs behind the main results and ablations, median wall-clock training time is 331 s (minimum 246 s for a plain policy, maximum 1115 s for the full `mocca` architecture, whose GRU encoder and consensus rounds are the most expensive forward pass in the ablation set), at a median throughput of 4.5×10^3 environment steps per second – the batched environment rather than the network is the bottleneck at this scale. The full set of runs reported in this paper – the credit and architecture ablations, the eight-surface scaling probe, and every additional probe in Section S5 – totals 12.7 h of GPU time; no result in this paper required infrastructure beyond a single workstation GPU.

Training convergence

Figure 2 gives the training-time behaviour behind every learned number in the main text: mean structural energy and divergence rate over training on the curriculum task distribution, averaged across seeds. Every variant is within a small margin of its asymptote by 750 000 of the 1 500 000 environment steps budgeted, confirming the training budget was chosen with headroom rather than tuned to convergence after the fact. `residual_only` starts substantially lower than every other variant and stays there for the first 200 000 steps, the training-time signature of the small-gain initialisation: it inherits the decentralised LQG base law’s performance from update one rather

Table 1: PPO and network hyperparameters.

Hyperparameter	Value
Parallel environments	128
Rollout length	128 steps
Epochs per update	4
Minibatches per epoch	4
BPTT segment length	32 steps
Learning rate (Adam)	3×10^{-4}
Discount γ	0.995
GAE λ	0.95
PPO clip range	0.2
Value loss coefficient	0.5
Entropy coefficient	2×10^{-3} (0 in the exploration sweep)
Max gradient norm	0.5
Curriculum fraction	0.4 of training
Hidden layer width (trunk)	128
Phasor encoder hidden width	64
Retained modes per phasor	2
Consensus rounds	2
Action-head output-layer gain (init.)	0.01

than the near-open-loop performance every other variant starts from. The divergence rate – a training statistic, not the evaluation metric of the main text’s results table – spikes above 0.5 % only in the first few thousand steps, before any policy has learned anything and while the curriculum is still easing in, and settles below 0.15 % thereafter for every variant: the curriculum keeps training itself stable even where the resulting policy later struggles on the harder end of the evaluation grid.

S2. Baseline controller synthesis

This section gives the synthesis equations behind every rung of the classical baseline ladder in the main text, so that “discrete-time LQG on local accelerometers” is reproducible from this document alone.

Augmented, discretised plant

Actuator lag is part of the design plant, not an unmodelled disturbance added afterwards: with $z = [q, \dot{q}, x_w, \delta]$ (the state of the equation of motion augmented with the deflections) and first-order servo lag of time constant τ_k per surface,

$$\dot{z} = \tilde{A}z + \tilde{B}u, \quad \tilde{A} = \begin{bmatrix} A & B \\ 0 & -\text{diag}(\tau_k^{-1}) \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} 0 \\ \text{diag}(\tau_k^{-1}) \end{bmatrix}, \quad (4)$$

where A, B are the plant matrices in state-space form and $u \in [-1, 1]^K$ is the normalised command. Every controller in this study is synthesised in discrete time at the 200 Hz control rate via the exact

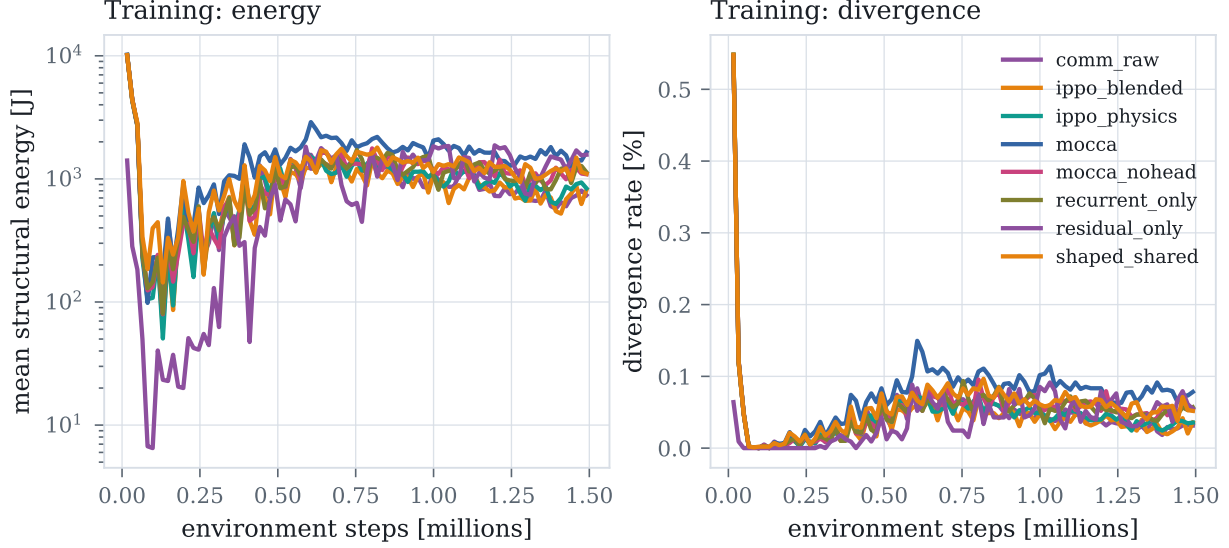


Figure 2: Training-time mean structural energy (left, log scale) and divergence rate (right) against environment steps, averaged over seeds, for every credit and architecture variant. All variants reach a stable plateau well inside the training budget; **residual_only** converges fastest because it starts from a decentralised LQG base law rather than from scratch.

zero-order-hold discretisation

$$\begin{bmatrix} \Phi & \Gamma \\ 0 & I \end{bmatrix} = \exp\left(\begin{bmatrix} \tilde{A} & \tilde{B} \\ 0 & 0 \end{bmatrix} \Delta t\right), \quad (5)$$

giving $z_{t+1} = \Phi z_t + \Gamma u_t$. Designing in continuous time and integrating the resulting gain with a forward-Euler step at $\Delta t = 5$ ms is not a minor approximation here: the wake states have $|\lambda|$ up to $\sim 1.3 \times 10^4$ rad s⁻¹, so $\Delta t |\lambda| \approx 63$, and the observer built that way is unstable regardless of how good its gain is.

LQR gain

The state cost weights physical structural energy rather than an arbitrary diagonal, plus a light penalty on held deflection so the optimum does not park surfaces at the travel limit:

$$Q = \begin{bmatrix} K_{\text{modal}} & & \\ & M_{\text{modal}} & \\ & & \text{diag}(w_{\text{rate}}/\delta_{\text{max},k}^2) \end{bmatrix}, \quad R = \text{diag}(w_{\text{effort}}/\delta_{\text{max},k}^2), \quad (6)$$

with $w_{\text{effort}} = 2 \times 10^4$, $w_{\text{rate}} = 10^2$ throughout. The discrete-time algebraic Riccati equation

$$P = \Phi^\top P \Phi - \Phi^\top P \Gamma (R + \Gamma^\top P \Gamma)^{-1} \Gamma^\top P \Phi + Q \quad (7)$$

gives the gain $\mathcal{K} = (R + \Gamma^\top P \Gamma)^{-1} \Gamma^\top P \Phi$ and command $u_t = -\mathcal{K} z_t$ (Anderson and Moore, 2007). **BatchedLQR** solves (7) once at a fixed design speed; **BatchedScheduledLQR** solves it on a 9-point airspeed grid over the evaluation envelope and linearly interpolates $\mathcal{K}$ between grid points; **BatchedJamAwareLQR** additionally zeros the failed surface's column of Γ per failure hypothesis and resolves (7) for each hypothesis on the same grid, so the surviving surfaces' gains already account for the loss rather than compensating for it reactively.

Local rate feedback

The floor of the ladder uses no plant model: with $\dot{h}_k, \dot{\alpha}_k$ the locally sensed plunge and twist rate at surface k 's own station,

$$u_k = -\left(g_h \dot{h}_k + g_\alpha \dot{\alpha}_k\right), \quad g_h = 0.35, \quad g_\alpha = 0.9, \quad (8)$$

clipped to $[-1, 1]$.

Steady-state Kalman observer on local accelerometers

The centralised and decentralised LQG rungs replace the true state z_t in $u_t = -\mathcal{K}\hat{z}_t$ with an estimate driven only by the accelerometer channels the agents themselves observe, $y_t = Hz_t$, where H is built from the acceleration rows of the plant rather than a selection matrix, since an accelerometer measures $\ddot{q}$, an algebraic function of the state rather than a state itself. The steady-state predictor-form filter is

$$\Sigma = \Phi\Sigma\Phi^\top + W - \Phi\Sigma H^\top \left(H\Sigma H^\top + V\right)^{-1} H\Sigma\Phi^\top, \quad (9)$$

$$L = \Phi\Sigma H^\top \left(H\Sigma H^\top + V\right)^{-1}, \quad (10)$$

$$\hat{z}_{t+1} = \Phi\hat{z}_t + \Gamma u_t + L(y_t - H\hat{z}_t), \quad (11)$$

with process noise $W = 10^{-4}I$ and measurement noise $V = 10^{-6} \text{diag}(\|H_i\|^2)$ scaled per-channel, since accelerometer rows of H have norms of order 10^4 – 10^5 and an unscaled V demands impossible measurement precision and drives L to blow up (Anderson and Moore, 2007). **BatchedLQG** solves (9)–(11) once with H built from all six channels and $\mathcal{K}$ commanding all three surfaces jointly, on the same speed grid as the scheduled LQR. **BatchedDecentralizedLQG** solves an *independent* instance of (9)–(11) per surface k , with H restricted to k 's own two accelerometer rows and Γ restricted to k 's own column, so agent k 's estimate, gain and command never depend on any other surface's measurement or actuation – the discrete-time, model-based analogue of the information structure the learned policies are given. Its scheduling grid spans the same $[1.0, 1.5] U_f$ range as the training curriculum exactly, by construction rather than coincidence, so the comparison in the main text is not confounded by the classical design being scheduled over a narrower envelope than the learned policy was trained on.

Re-tuning the decentralised LQG for control-authority margin

BatchedRobustDecentralizedLQG is structurally identical to **BatchedDecentralizedLQG**: same per-surface observer, same restriction to each agent's own two accelerometers and own actuator. Two remedies for its lack of a guaranteed margin (Doyle, 1978) were checked. The first, loop transfer recovery (Doyle and Stein, 1981), boosts the process-noise weighting W in (9) by $\rho b_k b_k^\top$ (agent k 's own reduced input column) as $\rho \rightarrow \infty$, pushing the filter loop toward the full-state loop shape. Sweeping ρ from 0 to 10^4 changes the control-authority margin (the largest reduction in true control authority, relative to the design assumption, the closed loop remains stable under, found by bisection on the discrete-time closed-loop spectral radius) not at all: it stays at 0.158 at $1.45 U_f$ throughout. Comparing against the same gain applied with perfect (observer-free) state feedback, whose margin is also 0.158, shows why: the observer is not the bottleneck, so there is nothing for an observer-side remedy to recover.

The second remedy, and the one **BatchedRobustDecentralizedLQG** actually uses, re-weights the LQR cost (6) itself: w_{effort} increased from 2×10^4 to 10^6 , with w_{rate} , W and V unchanged. This raises

the control-authority margin from 0.158 to 0.179 before plateauing (no further gain to $w_{\text{effort}} = 10^7$), at a cost to nominal decay rate: the closed-loop growth rate at design conditions worsens from 0.9744 to 0.9916 in discrete-time terms. `scripts/robust_baseline_check.py` reproduces both checks.

S3. Plant model supplementary material

Testbed parameters

Table 2 gives the geometric and structural parameters of the wing and Table 3 the actuator and sensing parameters, common to every controller in this study.

Table 2: Wing geometry and structural properties.

Parameter	Value
Semispan	6.096 m
Chord	1.8288 m
Mass per unit length	35.71 kg m ⁻¹
Polar inertia per unit length (about EA)	8.64 kg m
Bending stiffness EI	9.77×10^6 N m ²
Torsional stiffness GJ	0.987×10^6 N m ²
Elastic axis	33 % chord
Centre of gravity	43 % chord
Bending / torsion modes retained	2 / 2
Aerodynamic strips	24
Air density (sea level)	1.225 kg m ⁻³

Table 3: Control surface, actuator and sensing parameters (identical for all three surfaces).

Parameter	Value
Spanwise bands (fraction of semispan)	[0.05, 0.40], [0.40, 0.72], [0.72, 0.98]
Hinge location	75 % chord
Deflection travel	$\pm 15^\circ$
Rate limit	200 ° s ⁻¹
Actuator time constant	20 ms
Control update rate	200 Hz
Local observation per agent	2 accelerometers, plunge/twist rate, own saturation, air data

What flutter and its suppression look like

Every quantity in the main text is a scalar derived from a wing that is physically bending and twisting, worth seeing once before reducing it to decay rates. Figure 3 gives the same initial 5 cm tip pluck under two treatments at $U/U_f = 1.10$, six shared instants apart, on one shared deformation scale. The top row is what “flutter” names: bending and torsion lock into a fixed phase relationship and the response grows visibly within under a second, until the linear model is no longer valid. The bottom row is the identical pluck under centralised LQR: by the second frame the deflection

is already smaller than at $t = 0$, and stays there. Both are literal rollouts of the plant at the flight condition used throughout the main text’s results section. A supplementary animation of all four classical cases (open loop, centralised LQR, local rate feedback, and centralised LQR with the outboard surface jammed) is provided alongside the code.

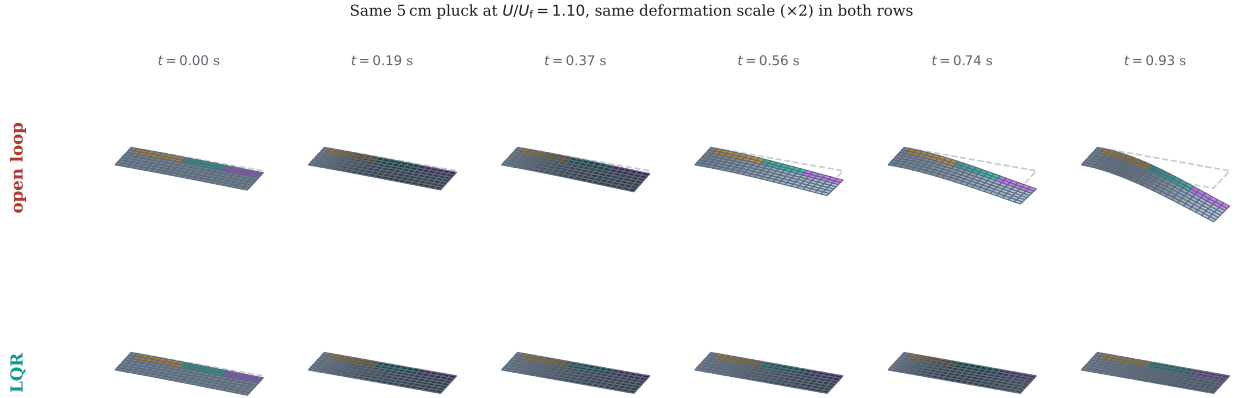


Figure 3: Six shared instants of the same 5 cm initial tip pluck at $U/U_f = 1.10$, one shared deformation scale (dashed outline: the undeformed reference planform). Top: open loop – bending and torsion coalesce and the response diverges within 0.93 s, at which point it leaves the linear regime and the rollout is stopped. Bottom: the identical pluck under centralised LQR, suppressed by the second frame and held near zero afterward.

Cross-validation against an unsteady vortex-lattice solver

The model buys speed with two approximations a reader is entitled to distrust: two-dimensional strip aerodynamics, and quasi-steady flap derivatives that neglect the flap’s own shed wake. We quantified both against SHARPy (del Carre et al., 2019), which couples an unsteady vortex-lattice method (Murua et al., 2012) to a geometrically exact beam. The comparison is clean because SHARPy’s own Goland template carries structural data identical to ours, so any disagreement is about the aerodynamics.

Flutter point SHARPy places flutter at 154.46 ms^{-1} and 69.3 rad s^{-1} at sea level, against 137.35 ms^{-1} and 69.34 rad s^{-1} for our model. The *frequency* agrees to 0.06 %, so both identify the same bending–torsion coalescence; the critical speed differs by -11.1 %. That direction is what two-dimensional aerodynamics predicts, since strip theory has no tip relief, over-predicts loading near the tip, and therefore under-predicts the flutter speed. Our plant is conservative, the safe direction for a control study.

Control authority This discrepancy runs the other way and is larger. A part-span flap loses effectiveness to spanwise carryover and to flow around its ends, neither of which a sectional derivative represents. Integrating our thin-airfoil $C_{l\delta} = 3.826$ per radian over the flapped span (25 % of span, read from the generated aerodynamic grid) gives a whole-wing $dC_L/d\delta$ of 0.957 per radian, against 0.495 from UVLM. Our plant over-estimates control authority by a factor of 1.9, grid-converged to within 3 % between 8×16 and 16×32 panels.

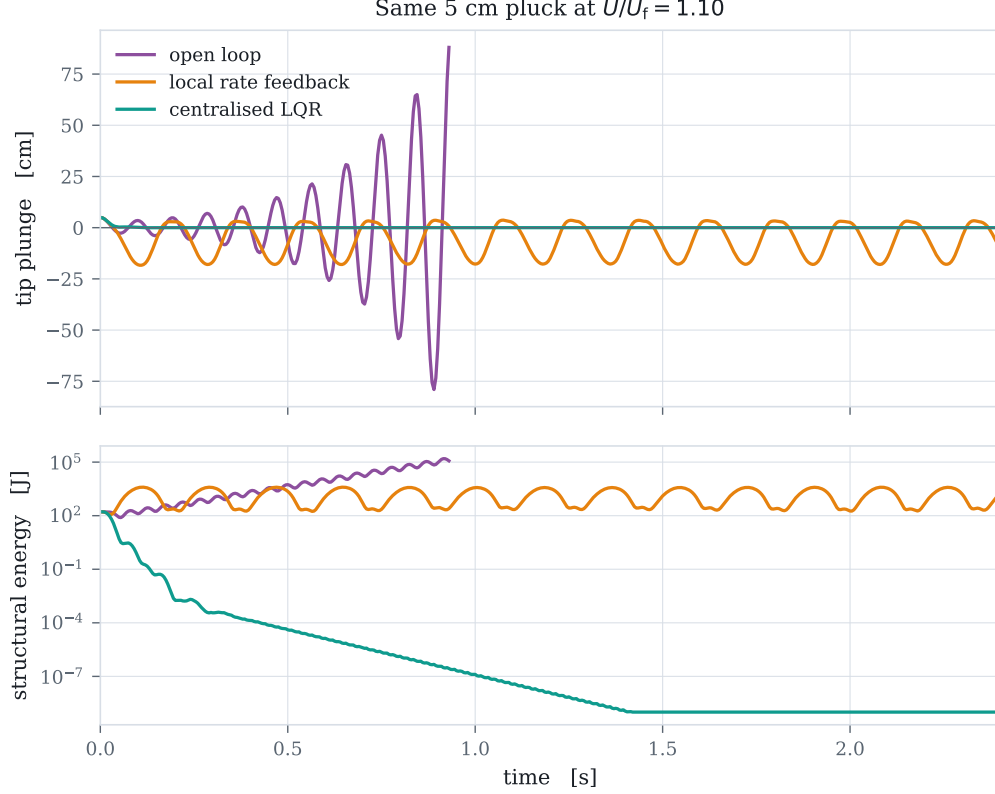


Figure 4: Tip plunge (top) and structural energy (bottom, log scale) for the same initial pluck under three of the five classical rungs. Local rate feedback neither grows nor decays: it damps local motion but never engages the unstable mode.

Consequence Every controller in this study, classical and learned, was evaluated on the same plant and received the same inflated authority, so relative comparisons, ablations and significance tests are unaffected. Absolute decay rates would not transfer unchanged to a UVLM plant, and we do not claim they would. The two discrepancies act in opposite directions – a harder flutter problem, an easier control problem – so they cannot be combined into a single safety factor.

S4. Related-work positioning

Table 4 summarises this paper against the literature it draws on, since no single prior study we are aware of combines more than one of these axes: multi-agent treatment of the individual control surfaces of one wing, a credit signal derived in closed form from the plant’s own physics rather than estimated, and validation of the plant itself against a higher-fidelity solver rather than only against a published linear benchmark.

S5. Additional probes

The results in the main text are held to a fixed standard: six seeds, Welch’s t -test, Cohen’s d and Hedges’ g , Holm-Bonferroni corrected. Several further questions arose directly out of those results

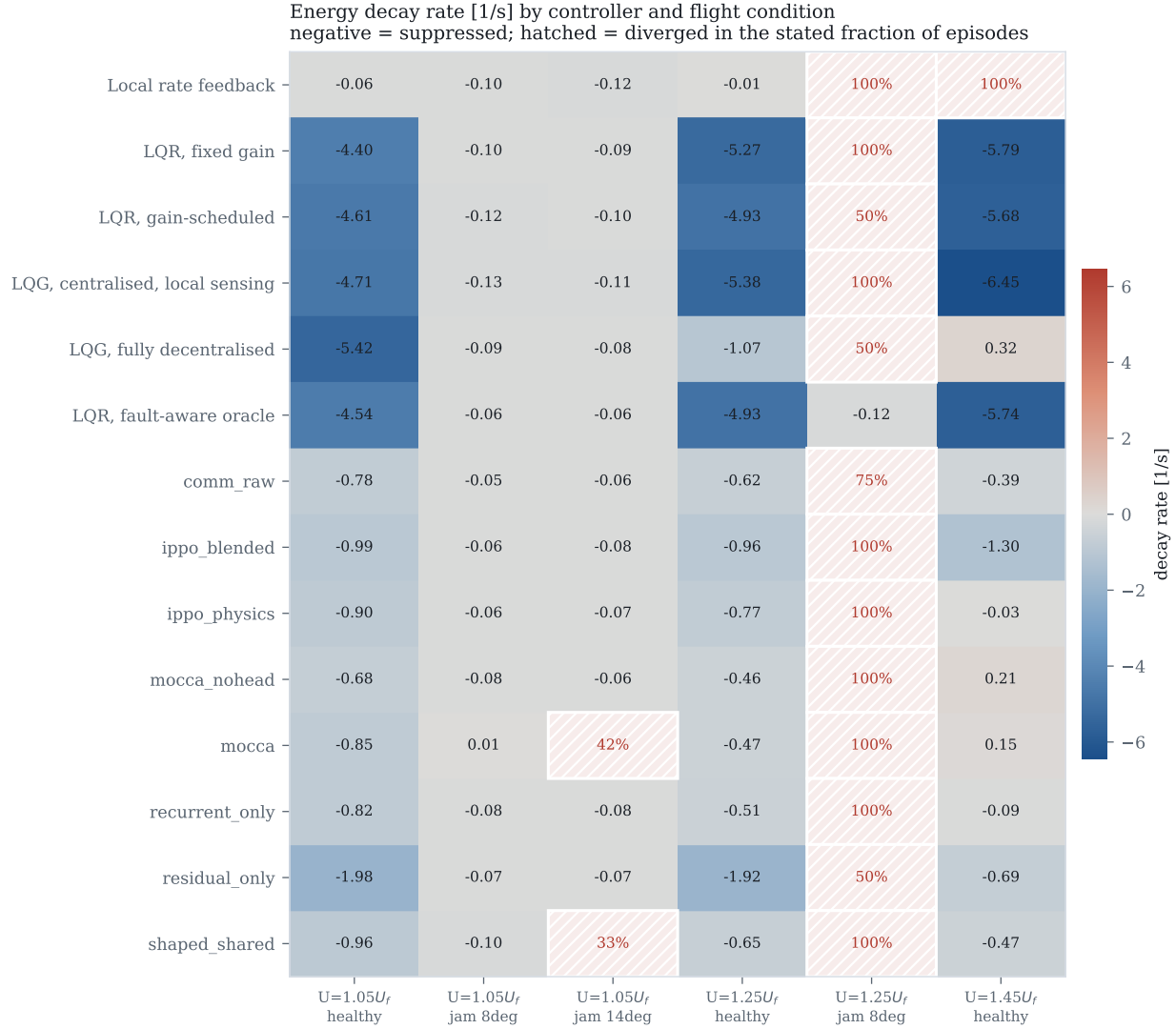


Figure 5: Decay rate by controller and condition. Hatching marks divergence. The same data as the main text’s results table, shown graphically.

and were checked, but not all to the same standard; they are reported here as probes rather than findings.

Exploration-scale variance at six seeds versus two An initial two-seed pilot of the spread behind the plateau discussed in the main text’s exploration-scale section suggested it fell sharply between the conventional and matched scale (0.132 at $\sigma = 0.30$ to 0.016 at $\sigma = 0.05$); the six-seed measurement reported in the main text does not confirm that (0.325 at $\sigma = 0.30$ to 0.257 at $\sigma = 0.05$), a second instance in this study of a two-seed spread understating the true variance, alongside the credit-versus-shaping correction in the credit-assignment ablation. The interior optimum and the factor-of-two effect on the mean are what survive at six seeds; the claim that the spread itself shrinks sharply at the matched scale does not, and the main text does not make it.

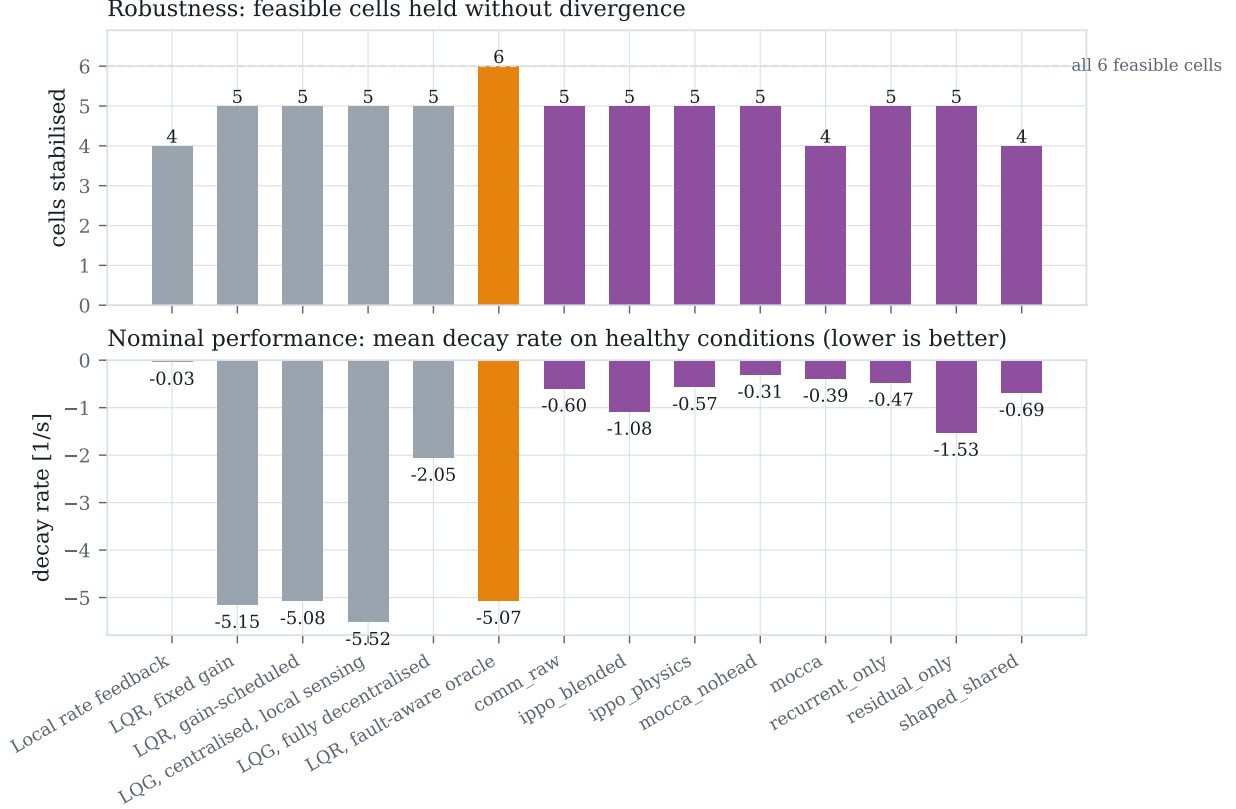


Figure 6: Every controller and variant on one axis. Top: feasible conditions held without divergence, out of the six the fault-aware oracle shows are stabilisable. Bottom: mean decay rate on healthy conditions. The learned variants (purple) match the classical rungs on robustness while trailing them on nominal damping, except for **residual_only**, which is the strongest learned result on both axes simultaneously.

Eight-surface scaling Section V of the main text reports that all nine evaluation conditions become feasible for the learned policies at eight surfaces. Table 5 gives the numbers behind that sentence, one or two seeds each at $\sigma = 0.05$; every healthy-decay value is listed per seed, not averaged, because two points do not estimate a standard deviation.

The negative architecture result survives moving to a residual policy The plain-policy architecture ablation found that consensus and raw neighbour sharing hurt. Applying the same two mechanisms to a policy that corrects a decentralised LQG base law instead of replacing it – the setting where **residual_only** is the strongest result in the paper at -1.53s^{-1} – gives **residual_comm** at -0.697 and -0.247 and **residual_health** at -0.518 and -0.066 : both clearly below **residual_only**, at both seeds. The negative result is not an artefact of the specific plain-policy setting it was first measured in.

Supervising the encoder on ground truth does not help **aux_encoder** adds a loss term supervising the phasor encoder directly on the true modal phasor – privileged information, unavailable outside simulation – rather than letting the encoder’s representation be shaped purely by the RL objective. Both seeds (-0.219 , -0.029) sit below the un-supervised **mocca_nohead** result

Table 4: This paper against the literature it draws on. A $\checkmark$ marks that the property holds for the representative citation(s) in that row; $-$ marks that it does not; n/a marks that the axis does not apply to that line of work at all (a single-agent or single-regulator method has no multi-agent credit decomposition to be closed-form *or* estimated, so neither $\checkmark$ nor $-$ would be meaningful there).

	Multi-agent surfaces	Closed-form credit	Validated against higher-fidelity solver
Classical AFS (Karpel, 1982; Waszak, 2001)	$-$	n/a	varies
Adaptive AFS (Downs and Prazenica, 2023; Mozaffari-Jovin et al., 2024)	$-$	n/a	$-$
Learned AFS (Li et al., 2025; Chen et al., 2023; Mu et al., 2022)	$-$	$-$	$-$
General MARL credit (Sune-hag et al., 2018; Rashid et al., 2018; Foerster et al., 2018)	$\checkmark$	$-$	n/a
Residual control (Johannink et al., 2019; Silver et al., 2018)	$-$	$-$	n/a
This paper	$\checkmark$	$\checkmark$	$\checkmark$

of -0.308 ± 0.169 , and the residual counterpart `aux_residual` (-0.431 , -0.510) sits far below `residual_only`’s -1.53 . We read this as mild evidence that the credit signal already supplies what the encoder needs to represent, and that an additional supervised loss competes with, rather than complements, the policy gradient here.

Eight surfaces changes what “held” means, not just how many The numbers behind “all nine conditions become feasible at eight surfaces” deserve a more careful reading than “held” alone provides: `mocca_nohead` holds nine of nine and eight of nine conditions, but at a healthy-condition decay rate of only -0.028 and -0.015 – an order of magnitude weaker than the three-surface result for the same architecture. `recurrent_only` holds every condition at both seeds, yet one seed’s healthy-condition decay is *positive* ($+0.135$). “Held” here means the episode did not cross the divergence threshold within its duration, not that the wing was driven to a decisively damped state; a bounded, barely-growing oscillation can hold a condition by this criterion without being a controller we would recommend. We stand by the qualitative claim that eight-surface architectures fail less catastrophically than three-surface ones, and retract any stronger reading of it than that.

S6. Sensitivity to control-influence identification error

The credit signal’s sign is decided by the modal residue $v_r^\top b_k$, which is free in simulation but requires an identified control-influence estimate on hardware. A wrong sign rewards a surface for exciting a mode rather than damping it, so before recommending the signal for anything beyond simulation we quantify, rather than assert, how much identification error it tolerates.

Table 5: Additional probes, one or two seeds each, at $\sigma = 0.05$. -- marks a probe not run for that variant.

Probe	Variant	Healthy decay [1/s], per seed	Held
<i>Do the negative mechanisms of the architecture ablation also hurt as a residual correction?</i>			
residual + raw comm.	<code>residual_comm</code>	-0.697, -0.247	6/9, 6/9
residual + health-channel consensus	<code>residual_health</code>	-0.518, -0.066	5/9, 5/9
<i>Does supervising the encoder on the true modal phasor help?</i>			
supervised encoder, plain head	<code>aux_encoder</code>	-0.219, -0.029	5/9, 5/9
supervised encoder, residual	<code>aux_residual</code>	-0.431, -0.510	5/9, 5/9
<i>Eight-surface scaling</i>			
consensus, plain head, $N=8$	<code>mocca_nohead</code>	-0.028, -0.015	9/9, 8/9
recurrent only, $N=8$	<code>recurrent_only</code>	+0.135, -0.011	9/9, 9/9

Analytic: how large an error flips a sign Modelling identification error as an independent relative perturbation on each (*mode, surface*) entry of B , scaled to the natural magnitude of that mode’s own row (entries in the same row share the same modal projection but each surface’s own aerodynamic derivative is independently estimated in practice), the most fragile entry – the mid-span surface’s (`aileron_mid`) residue on the first torsion mode – flips sign at just 11 % relative error, essentially identical across the flight envelope (11.0 % to 11.0 % at $1.05\text{--}1.45 U_f$, since the fragility ratio is speed-invariant by construction). Every other entry is more robust, ranging up to entries that do not flip within $\pm 100\%$ error at all. This is the honest answer to whether error in B threatens the credit signal: for the least robust surface–mode pairing, yes, at an identification error smaller than many real aerodynamic-derivative uncertainties; for the rest, no.

Under a *uniform* per-surface scale error instead – every entry of a surface’s column moving by the same fraction, which is what a simple overall effectiveness mis-estimate would look like – every sign flips simultaneously at exactly -100% relative error (classical control reversal), independent of magnitude. The plant’s own reported UVLM control-authority discrepancy (Section S3) is $+90\%$, an overestimate, which moves the assumed value *away* from zero rather than toward it: a safety factor of roughly $2\text{--}3\times$ in relative-error terms from the boundary that would matter, in the direction that already occurred.

Empirical: training with a misinformed credit signal We retrained the exact-plus-shaping credit variant with the true dynamics held fixed and only the credit signal’s belief about control authority perturbed by a uniform $\pm 60\%$, three seeds each. At the unperturbed baseline the healthy decay rate is -0.441 ± 0.330 ; believing 60 % more authority gives -0.259 ± 0.217 , and 60 % less gives -0.570 ± 0.131 . Neither direction collapses performance or reverses sign, though the spread across all three conditions at three seeds is wide enough that we do not read a monotone trend into the ordering. Both perturbations stay well short of the -100% sign-reversal boundary the uniform-scale analysis above predicts, so this is evidence that moderate uniform identification error degrades gracefully rather than catastrophically, not a test of what happens past that boundary.

References

- Brian D. O. Anderson and John B. Moore. *Optimal Control: Linear Quadratic Methods*. Dover Publications, 2007.
- Zhen Chen, Zhiwei Shi, Sinuo Chen, Shengxiang Tong, and Yizhang Dong. Active flutter suppression for a flexible wing model with trailing-edge circulation control via reinforcement learning. *AIP Advances*, 13(1):015317, 2023. doi: 10.1063/5.0130370.
- Alfonso del Carre, Arturo Muñoz-Simón, Norberto Goizueta, and Rafael Palacios. SHARPy: A dynamic aeroelastic simulation toolbox for very flexible aircraft and wind turbines. *Journal of Open Source Software*, 4(44):1885, 2019. doi: 10.21105/joss.01885.
- Patrick S. Downs and Richard J. Prazenica. Adaptive control of a flexible wing for flutter suppression and disturbance rejection. In *AIAA SCITECH 2023 Forum*, 2023. doi: 10.2514/6.2023-0130.
- John C. Doyle. Guaranteed margins for LQG regulators. *IEEE Transactions on Automatic Control*, 23(4):756–757, 1978. doi: 10.1109/TAC.1978.1101812.
- John C. Doyle and Gunter Stein. Multivariable feedback design: Concepts for a classical/modern synthesis. *IEEE Transactions on Automatic Control*, 26(1):4–16, 1981. doi: 10.1109/TAC.1981.1102555.
- Jakob N. Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients. In *AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11794.
- Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium*, 2015. doi: 10.48550/arXiv.1507.06527.
- Tobias Johannink, Shikhar Bahl, Ashvin Nair, Jianlan Luo, Avinash Kumar, Matthias Loskyll, Juan Aparicio Ojea, Eugen Solowjow, and Sergey Levine. Residual reinforcement learning for robot control. In *IEEE International Conference on Robotics and Automation*, 2019. doi: 10.48550/arXiv.1812.03201.
- Mordechay Karpel. Design for active flutter suppression and gust alleviation using state-space aeroelastic modeling. *Journal of Aircraft*, 19(3):221–227, 1982. doi: 10.2514/3.57379.
- Jinying Li, Yuting Dai, Yating Hu, Yuming Zhang, and Chao Yang. Deep reinforcement learning control for stall flutter via active camber morphing. *Physics of Fluids*, 37(10):105166, 2025. doi: 10.1063/5.0295770.
- Sina Mozaffari-Jovin, Roohollah D. Firouz-Abadi, and Jafar Roshanian. L1 adaptive aeroelastic control of an unsteady flapped airfoil in the presence of unmatched nonlinear uncertainties. *Journal of Sound and Vibration*, 578:118334, 2024. doi: 10.1016/j.jsv.2024.118334.
- Xusheng Mu, Rui Huang, Qitong Zou, and Haiyan Hu. Machine learning-based active flutter suppression for a flexible flying-wing aircraft. *Journal of Sound and Vibration*, 529:116916, 2022. doi: 10.1016/j.jsv.2022.116916.
- Joseba Murua, Rafael Palacios, and J. Michael R. Graham. Applications of the unsteady vortex-lattice method in aircraft aeroelasticity and flight dynamics. *Progress in Aerospace Sciences*, 55: 46–72, 2012. doi: 10.1016/j.paerosci.2012.06.001.

- Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, 2018. doi: 10.48550/arXiv.1803.11485.
- John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016. doi: 10.48550/arXiv.1506.02438.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347.
- Tom Silver, Kelsey Allen, Josh Tenenbaum, and Leslie Kaelbling. Residual policy learning. *arXiv preprint arXiv:1812.06298*, 2018. doi: 10.48550/arXiv.1812.06298.
- Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning. In *International Conference on Autonomous Agents and MultiAgent Systems*, 2018. doi: 10.48550/arXiv.1706.05296.
- Martin R. Waszak. Robust multivariable flutter suppression for benchmark active control technology wind-tunnel model. *Journal of Guidance, Control, and Dynamics*, 24(1):147–153, 2001. doi: 10.2514/2.4694.
- Chao Yu, Akash Velu, Eugene Vinitzky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*, 2022. doi: 10.48550/arXiv.2103.01955.